ORBIS

Master Project Specification

An Intelligent Context-Aware Desktop Agent for Workspace Continuity and Autonomous Computer Assistance

Target: Windows 11 desktop | UI: Electron + React | Gateway: FastAPI | Memory: SQLite + FAISS + Workspace Knowledge Graph

This document is the functional blueprint for what ORBIS should do, how users should command it, how tasks should execute, how continuity should work, how restoration should work, what the interface should show, and what ORBIS must never do.

1. Product Definition

ORBIS is a general-purpose, local-first desktop AI agent. A user should be able to speak or type a natural-language request and ORBIS should determine whether the request needs an answer, research, file access, coding, terminal execution, document generation, system control, workspace retrieval, restoration, or a multi-step combination of these capabilities.

The central research differentiator is workspace continuity. ORBIS should reconstruct task-level context across applications, files, browser resources, events, sessions, intent, and relationships so that a user can return later and continue meaningful work instead of starting from zero.

General-purpose does not mean unrestricted. ORBIS can perform an authorized digital task only when an implemented and safety-controlled capability exists.

2. Core User Experience

- User gives a request.
- ORBIS resolves conversational references and current context.
- ORBIS determines the intent and whether fresh information is required.
- ORBIS grounds the request using current workspace and authorized historical context.
- ORBIS creates a plan for multi-step work.
- Safety policy validates the plan.
- Approval is requested for actions requiring it.
- Registered tools execute the plan.
- Live events are streamed to the UI.
- Postconditions are verified.
- Relevant results are recorded for continuity.
- ORBIS explains what happened, what could not be done, and what can happen next.

3. What ORBIS Should Do

- Answer normal questions.
- Explain technical concepts at beginner or technical depth.
- Research current information and provide sources.
- Compare sources and identify disagreements.
- Inspect approved files and folders.
- Create supported documents and project artifacts.
- Read and summarize project files.
- Run controlled terminal commands.
- Assist with coding and debugging.
- Observe authorized workspace activity.
- Segment activity into meaningful work sessions.

- Create Context DNA for sessions.
- Build a workspace knowledge graph.
- Perform semantic and hybrid continuity retrieval.
- Infer unfinished intent conservatively.
- Restore supported applications, files, URLs, windows, and layouts.
- Provide push-to-talk voice interaction.
- Control supported system functions such as audio.
- Show real-time task execution and verification.
- Ask for clarification when the request is ambiguous.
- Refuse or block unsafe actions rather than pretending.

4. What ORBIS Must Not Do

- No keylogging.
- No password harvesting.
- No credential scraping.
- No unrestricted surveillance.
- No hidden microphone recording.
- No unrestricted clipboard capture.
- No arbitrary bypass of OS security.
- No fake execution results.
- No fake research sources.
- No silent high-risk actions.
- No direct unrestricted tool-execution API.
- No claim of perfect restoration.
- No claim of certainty when continuity evidence is weak.

5. System Architecture

6. Tool Catalog

7. Command Categories

7.Basic Q&A

- What can you do?
- What is TCP?
- Explain virtual memory.
- Why does DNS cache results?
- Give me a beginner explanation.
- Give me the technical explanation.
- Summarize that.
- Compare TCP and UDP.

7.Research

- Research the latest developments in context-aware desktop agents.
- Find official documentation for FastAPI.

- Compare three sources about desktop AI agents.
- Research this topic and give me evidence.
- Find conflicting claims and explain them.
- Research X and create a Markdown report.

7.Files

- List the files in my workspace.
- Find the README.
- Read this file.
- Summarize this folder.
- Compare these two files.
- Create report.md.
- Save this result in the project.
- Show which files were involved in yesterday's work.

7.Coding

- Explain this code.
- Find the bug.
- Review this file.
- Add a test.
- Implement the safe fix.
- Run the tests.
- Explain why the build failed.
- Refactor this without changing behavior.

7.Terminal

- Run the tests.
- Run the frontend build.
- Run the backend tests.
- Show git status.
- Show git diff.
- Check the Python version.
- Check the Node version.
- Start the development server.

7.Continuity

- What was I working on?
- What was I working on yesterday?
- What changed today?
- Which project was active?
- Which files did I use?
- What task was unfinished?
- What should I do next?
- Continue where I left off.

7.Restoration

- Restore my workspace.

- Restore yesterday's session.
- Open the project where I left off.
- Restore the last development session.
- Restore only the editor and terminal.
- Restore the workspace but do not launch the browser.

7.System

- Show the current volume.
- Set volume to 40%.
- Increase volume by 10%.
- Decrease volume.
- Mute the system.
- Unmute the system.
- Open supported system settings.

7.Documents

- Create a project report.
- Create a README.
- Turn these notes into a report.
- Create a research summary.
- Create a comparison report.
- Save it as Markdown.

7.Voice

- Hey ORBIS, what can you do?
- Research X.
- Open my project.
- What was I working on?
- Continue where I left off.
- Set volume to 40%.
- Stop.
- Cancel that.

8. Natural-Language Semantics

- 'latest' means fresh retrieval when current information matters.
- 'yesterday' means the user's local date and the stored session timeline.
- 'that' refers to the most recent unambiguous artifact or topic.
- 'continue' means identify the most relevant unfinished work before acting.
- 'do it' refers to the current pending plan only if the reference is unambiguous.
- If multiple files/resources match, ask instead of guessing.
- If the user changes topic, preserve useful artifacts but change active task focus.
- 'stop' or 'cancel' should terminate pending execution as safely as possible.

9. Conversation Context

ORBIS must maintain bounded multi-turn context. It should track recent turns, resolved references, current task, current artifacts, selected resources, and active view. It should not retain an unlimited conversation history as the sole memory mechanism.

10. Workspace Observation

Observation is passive and privacy-conscious. It provides evidence for continuity rather than acting as surveillance.

- Observe supported active application/window state.
- Observe approved file/workspace events.
- Store timestamps and task-related metadata.
- Apply protected path and privacy exclusions.
- Do not capture passwords.
- Do not keylog.
- Do not persist raw audio.
- Do not capture unrestricted screenshots.
- Do not expose protected filenames through directory listing.

11. Sessionization

Raw events must be grouped into coherent work sessions. Sessionization should consider time, active application continuity, project/resource relationships, event sequences, and task signals.

A session is a meaningful unit of work, such as developing ORBIS, researching a topic, preparing a report, or debugging a build. It is not simply an arbitrary time interval.

12. Context DNA

13. Knowledge Graph

The graph is functional memory infrastructure, not merely a visualization.

14. Semantic Memory and Hybrid Retrieval

Semantic embeddings and FAISS provide similarity retrieval. ORBIS should combine semantic similarity with lexical overlap, recency, project relevance, application continuity, and graph proximity.

A semantically similar old session must not automatically outrank a recent, project-matching session. Retrieval should expose evidence and confidence.

15. Intent Continuity Engine

ICE should answer three questions: what was the user trying to accomplish, whether the work appears complete, and what action could logically continue it.

- Use recent workspace events.
- Use prior task language.
- Use file/resource state.
- Use application continuity.
- Use temporal gaps.
- Use completion evidence.
- Do not convert weak evidence into certainty.
- Explain uncertainty when appropriate.

16. Restoration

- Receive restoration request.
- Retrieve candidate sessions.
- Rank candidate context.

- Show or construct context summary.
- Generate a declarative restoration plan.
- Validate applications, paths and URLs.
- Preflight resources.
- Classify risk.
- Request approval when required.
- Execute supported actions.
- Bind launched applications to verified windows where supported.
- Arrange supported windows/layouts.
- Isolate individual failures.
- Verify outcomes.
- Brief the user on what was restored and what was not.

Restoration does not guarantee exact internal application state, authenticated browser sessions, unsaved UI state, private application state, or arbitrary UI controls.

17. Safety Model

Deterministic safety rules are authoritative. The LLM cannot override them.

18. Approval

- Show exactly what ORBIS plans to do.
- Show the target resource.
- Explain why approval is required.
- Show reversibility when known.
- Provide Approve, Reject and Cancel.
- Never execute a high-risk step while approval is pending.

19. Task State Machine

Canonical lifecycle:

IDLE → REQUEST_RECEIVED → NORMALIZING → CONTEXT_GROUNDING → INTENT_CLASSIFIED → PLAN_CREATED → SAFETY_CHECK → APPROVAL_REQUIRED/EXECUTING → STEP_STARTED → TOOL_EXECUTION → VERIFICATION → STEP_COMPLETED → next step → TASK_COMPLETED. Failures may transition to REPLAN or TASK_FAILED; rejection transitions to TASK_CANCELLED.

20. Verification Rules

A successful process exit code is not automatically proof that the user's intended outcome occurred.

21. Web Research

- Search only when needed.
- Use bounded search/fetch.
- Validate URLs and redirects against SSRF policy.
- Treat external pages as untrusted input.
- Never obey instructions embedded in external pages as ORBIS policy.
- Preserve evidence/source attribution.
- Distinguish snippets from fetched content.
- Do not claim current information without current retrieval.

22. Terminal

- shell=False.
- Executable allowlist.
- Validate executable path.
- Workspace boundary.
- Bound output.
- Secret redaction.
- Dynamic risk.
- Postcondition verification.
- No arbitrary shell escape.

23. Files

- Normalize paths.
- Reject traversal.
- Protect sensitive paths.
- Filter protected filenames.
- Redact secrets.
- Handle symlinks conservatively.
- Do not overwrite without required policy.
- Use explicit supported document formats.

24. Voice

Push-to-talk → SoundDevice capture → local Faster-Whisper STT → conversation normalization → normal ORBIS task pipeline → final response → Windows SAPI5 TTS.

- Raw audio is not persisted by default.
- Voice and typed commands share the same safety model.
- Cancel must be safe.
- Voice should surface final results, not hidden reasoning.

25. UI Screens

26. Visual Design Rules

- Dark obsidian/near-black foundation.
- Minimal and cinematic.
- Generous negative space.
- Subtle glass surfaces.
- Controlled motion.
- Globe is the primary identity.
- OptionWheel is primary navigation.
- No permanent generic SaaS sidebar.
- No excessive neon glow.
- No fake metrics.
- React Bits components support the product rather than becoming the product.

27. Current Visual Component Map

28. Multi-Step Workflows

29. Failure Handling

- Never show success when the task failed.
- Identify the failed step.
- Preserve successful prior steps.
- Offer recovery.
- Do not repeat destructive actions automatically.
- Explain missing resources.
- Explain authentication requirements.
- Explain unsupported capabilities.
- Ask clarification when confidence is low.

30. Research Evaluation

Evaluation claims must be measured experimentally; they should not be presented as achieved results before testing.

31. Baselines and Ablations

- Keyword-only retrieval.
- Metadata-only retrieval.
- Semantic vector retrieval.
- Semantic plus graph retrieval.
- Full hybrid retrieval.
- Remove recency.
- Remove graph.
- Remove intent summary.
- Remove ICE.
- Remove restoration verification.

32. Demonstration Scenarios

Demo 1 — Q&A

Ask a technical question and receive a clear answer without unnecessary tool use.

Demo 2 — Research

Ask for current information; show sources and evidence.

Demo 3 — Document

Turn notes into a saved report and verify the artifact.

Demo 4 — Terminal

Run tests through the controlled terminal pipeline and report output.

Demo 5 — Continuity

Return after work and ask what you were doing; show real Context DNA/session evidence.

Demo 6 — Restoration

Request yesterday's workspace; show candidate context, plan, approval and restoration result.

Demo 7 — Voice

Use push-to-talk, receive local transcription, execute the request, hear the response.

Demo 8 — Safety

Attempt a high-risk or prohibited operation and show the approval/blocking behavior.

33. Extensibility Rule

Every new capability should be introduced as a controlled tool with a schema, risk classification, validation, execution method, verification method, tests, and UI exposure. If the capability does not exist, ORBIS must say so rather than simulating it.

- Future email/calendar integrations require explicit authorization.
- Browser automation requires its own safety model.
- Office formats require supported libraries and verification.
- Repository actions require explicit scope and approval.
- New OS controls require dedicated testing.
- External integrations require license and security review.

34. Developer Acceptance Checklist

- Globe remains unchanged.
- Awakening remains unchanged.
- OptionWheel remains primary navigation.
- All major views remain reachable.
- Tool execution uses the authoritative pipeline.
- No fake data in functional flows.
- Backend tests pass.
- Frontend build passes.
- Electron verification passes.
- No console errors.
- Continuity remains evidence-grounded.
- Restoration uses the existing restoration engine rather than duplicating it.
- Approval remains authoritative.
- Verification exists for consequential actions.
- Git changes are attributable before committing.

35. Final Product Definition

ORBIS should be experienced as one coherent assistant, not a chatbot plus unrelated utilities. The agent should understand a request, connect it to the user's current or historical workspace, decide what tools are required, safely execute authorized steps, verify the outcome, explain the result, and preserve enough context that the next interaction can begin from where the previous work ended.

The most important loop is: WORK → OBSERVE → UNDERSTAND → REMEMBER → RETURN → RECONSTRUCT → RESTORE → CONTINUE.

The project is successful when ORBIS can demonstrate that loop truthfully with real workspace data, real task execution, real safety boundaries, and real verification.

Appendix A — Golden Commands

- Hey ORBIS, what can you do?
- Explain this.
- Research the latest information about X.
- Research X, compare the sources, and create a report.
- List my project files.
- Read this file and summarize it.
- Run my tests and explain the failure.
- Fix the safe issue and rerun the tests.
- What was I working on?
- What was I working on yesterday?
- What task was unfinished?
- Continue where I left off.
- Restore my workspace.
- Restore only the editor and terminal.
- Set volume to 40%.
- Show my current volume.
- Open the project where I left off.
- Stop.
- Cancel that.

Appendix B — Non-Goals

- Perfect restoration of arbitrary application internals.
- Guaranteed authenticated browser-session recovery.
- Perfect human-intent inference.
- Unrestricted OS automation.
- Unrestricted surveillance.
- Silent high-risk execution.
- Fake UI success.
- Claiming every third-party application is fully automatable.

Appendix C — One-Sentence Definition

ORBIS is a local-first, safety-controlled desktop AI agent that understands natural-language and voice requests, executes authorized digital tasks through verified tools, and reconstructs prior workspace context so users can continue work instead of restarting it.

Layer	Responsibility
Electron + React	Desktop shell, visual interface, navigation, interaction.
FastAPI Gateway	Authoritative application API and WebSocket gateway.
Agent Orchestrator	Coordinates planning, tools, safety, approval, verification.
Conversation Context	Bounded multi-turn references and active artifacts.
Hierarchical Planner	Breaks multi-step requests into validated steps.
Intent Classifier	Determines task category and execution needs.
Safety / Policy	Risk classification, blocking, approval requirements.
Tool Registry	Authoritative list of capabilities and schemas.
Verification / Critic	Checks postconditions and detects failed outcomes.
SQLite	Events, sessions, task history and durable local state.

Layer	Responsibility
FAISS + embeddings	Semantic retrieval.
Knowledge Graph	Relationships among sessions, tasks, resources and events.
Continuity Engine	Context reconstruction.
ICE	Intent continuity and unfinished-work inference.
Restoration Engine	Validated workspace reconstruction.

Tool	Should do	Typical commands
FileTool	Safe file inspection	list files; read file; find resource; summarize file
DesktopTool	Supported desktop observation/actions	active app; open supported resource
TerminalTool	Controlled terminal	run tests; build; git status; version checks
WebResearchTool	Fresh research	research topic; compare sources; find docs
ContentGenerator	Generate content	write report; draft README; synthesize notes
DocumentTool	Persist supported artifacts	create Markdown/TXT; verify artifact
KnowledgeTool	Answer/knowledge retrieval	explain concept; answer technical question
ContinuityTool	Retrieve prior work	what was I working on; previous session
IntentContinuityTool	Infer unfinished work	what was unfinished; what next
RestorationTool	Restore workspace	restore yesterday; restore project
SystemControlTool	System controls	read/set volume; supported system actions
SystemUtilityTool	System information/utilities	system status and supported utilities
VoiceTool	Voice interaction	listen; transcribe; speak
EchoTool	Diagnostics	safe pipeline test

Conversation	Expected interpretation
Open the report.	Select the unique recent report if unambiguous.
Summarize it.	Use the report just selected.
Now create a PDF.	Use the active report if PDF creation is supported.
Compare it with the README.	Use the current report and project README.
Use the second source.	Resolve to the second source in the active research result.

Element	Purpose
Intent	Likely user goal.
Applications	Programs involved.
Files	Relevant resources.
Browser resources	Relevant web resources when available.
Time	When the work occurred.
Relationships	Semantic and causal links.
Confidence	Strength of evidence.
Privacy policy	What may be reused.
Project	Associated project/workspace.
Events	Evidence supporting the context.

Relationship	Meaning
contains	Session contains resource/event.
used_in	Resource used in task/session.
opened	Application opened resource.
related_to	Semantic relationship.
followed_by	Temporal continuation.
modified_during	Resource changed during session.
supports_task	Resource supports task.

Risk	Examples	Expected behavior
LOW	Read file, inspect status	Execute if policy permits.
MEDIUM	Controlled terminal action, noncritical modification	May require confirmation.
HIGH	Overwrite important artifact, consequential system action	Explicit approval.
BLOCKED	Credential harvesting, prohibited surveillance, unsafe bypass	Reject.

Event	Meaning
TASK_STARTED	Task accepted.
PLAN_CREATED	Plan generated.
STEP_STARTED	Step became active.
TOOL_EXECUTION	Tool is running.
VERIFICATION_STARTED	Postcondition check began.
VERIFICATION_COMPLETED	Verification ended.
STEP_COMPLETED	Step succeeded.
TASK_COMPLETED	Whole task succeeded.
TASK_FAILED	Task failed.
APPROVAL_REQUIRED	User approval needed.

Operation	Verification
Create file	File exists and content is readable.
Research	Sources/evidence exist.
Terminal build	Process result plus expected build condition.
Set volume	Read back volume when supported.
Restore window	Expected process/window exists.
Continuity	Returned context has stored evidence.
Document	Output exists and is readable.

View	Required behavior
Home / ORBIS Core	Large central Globe, left OptionWheel, readiness state, restrained ambient effects.
Command	Conversation, SplitText/TextType, real task state, Stepper, Fluid Glass contextual inspector, contextual Dock, approvals.
Live Flow	Real WebSocket events, LogoLoop, Stepper, Fluid Glass event/metrics panel, contextual Dock.
Workspace	Observation telemetry, Context DNA, session timeline, events, Folder, AnimatedList, TiltedCard, graph telemetry.
Tools	Real registry-backed tools, categories, risk, schema, inspector, safe Try Prompt.
Restoration	Candidate sessions, context summary, plan, preflight, approval, execution, per-resource result, verification.
System Control	Real volume, ElasticSlider, supported controls, telemetry.
Research / Documents	Research input, sources, evidence, synthesis, artifact creation and verification.
Settings	Voice preferences, UI motion preference, privacy/safety visibility, gateway/model status.

Component	Role	Status in current plan
Globe	Home / ORBIS identity	Implemented / locked
DecryptedText	Awakening	Implemented / locked
SplitText	Conversation	Implemented
LogoLoop	Live Flow / Research	Implemented
OptionWheel	Primary navigation	Implemented / locked

Component	Role	Status in current plan
Specter Orb	Ambient state	Unavailable without required Pro source
Agentic Ball	Agent activity	Unavailable without required Pro source
TextType	Conversation/status	Implemented
Folder	Workspace	Implemented
ElasticSlider	Controls/settings	Implemented
AnimatedList	Activity/history	Implemented
Dock	Contextual tools	Implemented
Stepper	Task/workflow	Implemented
Fluid Glass	Contextual panels	Implemented
Tilted Card	Contextual information	Implemented

Workflow	Stages
Research → Report	Search → fetch → extract → compare → synthesize → generate → save → verify.
Debugging	Inspect → test → parse failure → inspect code → propose fix → approval if needed → modify → retest → verify.
Workspace continuation	Retrieve sessions → rank → identify intent → explain evidence → ask whether to restore → restore if requested.
Restoration	Retrieve → plan → preflight → approval → launch/open/arrange → verify → report.
Document generation	Gather sources → generate content → write artifact → verify file.
File comparison	Resolve files → read → compare → summarize.
Coding task	Inspect architecture → plan → modify authorized files → test → verify → summarize.
System control	Read current state → plan update → execute → read back → report.

Metric	Purpose
Context retrieval precision	Correctness of retrieved context.
Context retrieval recall	Coverage of relevant context.
Intent continuity accuracy	Correctness of unfinished-work inference.
Restoration success	Supported resources restored successfully.
Plan validity	Plans passing deterministic validation.
Verification accuracy	Correct success/failure detection.
Latency	Request-to-result time.
Approval correctness	High-risk actions gated correctly.
Failure containment	Independent failures isolated.